

Лямбды в C++

Замыкания, захват, generic lambdas, type erasure

Пучкин Станислав (СПбГУ)

C++ Семинар

15 апреля 2026 г.

```
1 // Без захвата: анонимная структура + приведение к func*
2 auto adder = [](int x, int y) { return x + y; };
3
4 // Поэтому работает неявное приведение:
5 int (*pf)(int, int) = adder;          // ok
6
7 // Тип уникален - переписать нельзя:
8 auto f = []{};
9 f = []{}; // ошибка: несовместимые типы
10
11 // Unary+ форсирует приведение к func* (positive hack):
12 auto g = +[]{};
13 g = []{}; // ok - g теперь указатель на функцию
```

Что и как захватывается

```
1  int g = 1;
2  void foo() {
3      int x = 10; static int s = 2;
4
5      auto lam = [=] { return x + s + g; };
6      auto lam2 = [&] { return x + s + g; };
7      auto lam3 = [&x, copy_s=s, copy_g=g] {
8          return x + copy_s + copy_g;
9      };
10     x = 99;
11     std::cout << lam() << std::endl; // 13
12     std::cout << lam2() << std::endl; // 102
13     std::cout << lam3() << std::endl; // 102
14     s = 10;
15     std::cout << lam() << std::endl; // 21
16     std::cout << lam2() << std::endl; // 110
17     std::cout << lam3() << std::endl; // 102
18 }
19 // Правило: захватывается только локальный
20 // нестатический контекст. static и глобальные
21 // доступны напрямую и изменения в них всегда видны.
```

```
1 #include <iostream>
2
3 auto foo(int x = 10) {
4     return [&]{return x;};
5 }
6
7 int main() {
8     auto f = foo();
9     std::cout << f(); // 32766
10    return 0;
11 }
```

Взятие по ссылке в многопоточном коде

```
1 void byref() {
2     std::vector<std::thread> threads;
3     int i;
4     for (i = 0; i < 10; i++) {
5         threads.emplace_back([&i] {
6             std::cout << i << '␣'; });
7     }
8     for (auto &th : threads) { th.join(); }
9     // 2 6 7 7 8 99 10 10 10
10    std::cout << '\n';
11 }
12
13 void bycopy() {
14     std::vector<std::thread> threads;
15     for (int i = 0; i < 10; i++) {
16         threads.emplace_back([i] {
17             std::cout << i << '␣'; });
18     }
19     for (auto &th : threads) { th.join(); }
20     // 0 1 2 3 4 5 6 7 9 8
21     std::cout << '\n';
22 }
```

```
1 // [a] → operator() const: нельзя менять копию
2 // [a]() mutable → без const: можно
3 auto counter = [a = 0]() mutable { return ++a; };
4 // counter() → 1, counter() → 2, ...
5
6 // Захват по const-ссылке -: нужно переименование (C++14)
7 int x = 5;
8 auto lmd = [&rx = std::as_const(x), vx = x] {
9     return vx + rx; // rx: const int&
10 };
11 // lmd() → 10, но rx нельзя изменить
12
13 // Move-захват (C++14 init-capture)
14 auto ptr = std::make_unique<int>(42);
15 auto f = [p = std::move(ptr)]() { return *p; };
16 // ptr == nullptr после move
17 // f() == 42
```

Лямбда - синтаксический сахар для анонимного функтора

```
1  struct ExpectedSimpleType {
2      static int func(int c) { return c; }
3      int operator()(int c) { return func(c); }
4      operator decltype(&func)() {return func;}
5  };
6
7  auto simple_lambda = [](int x) {return x;};
8  int (*simple_lambda_ptr)(int) = simple_lambda;
9  ExpectedSimpleType est;
10 int (*est_ptr)(int) = est;
11
12 using SimpleLambdaT = decltype(simple_lambda);
13 std::println("{} , {} ",
14     (*(SimpleLambdaT *)&est)(7),
15     (*(ExpectedSimpleType *)&simple_lambda)(8)); // 7, 8
```

Лямбда с захватом - анонимная структура-функтор

```
1  int main() {
2      struct ExpectedType {
3          int a; int &b;
4          ExpectedType(int a, int &b) : a(a), b(b) {}
5          auto operator()(int c) { a = -c; b = -(a + c); }
6      };
7
8      int b = 10;
9      auto lambda_function = [a = 0, &b](int c) mutable {
10         a = c; b = a + c;};
11      using LambdaT = decltype(lambda_function);
12      lambda_function(5);
13
14      int c = 2;
15      auto et = ExpectedType(0, c);
16
17      struct Derived : LambdaT {
18         Derived(LambdaT &t) : LambdaT(t) {}
19         // using LambdaT::operator();
20     } d(lambda_function);
21     d(5);
```


Лямбда с захватом - анонимная структура-функтор

```
1  std::println("a: {}, c: {}", et.a, c); // a: 0, c: 2
2  (*(LambdaT *)&et)(7);
3  std::println("a: {}, c: {}", et.a, c); // a: 7, c: 14
4
5  // a: 5, b: 10
6  std::println("a: {}, b: {}", *(int *)&lambda_function, b);
7  (*(ExpectedType *)&lambda_function)(7);
8  // a: -7, b: 0
9  std::println("a: {}, b: {}", *(int *)&lambda_function, b);
10
11 // typeid: Z4mainEUliE_
12 std::println("typeid: {}", typeid(LambdaT).name());
13 }
```

Лямбда с захватом - анонимная структура-функтор

```
1 $ gdb build/lambdaas
2 (gdb) b main
3 Breakpoint 1 at 0x45b4: file ../lambdaas.cpp, line 5.
4 (gdb) run
5 Breakpoint 1, main () at ../lambdaas.cpp:5
6 5      int main() {
7 (gdb) n
8 13      int b = 10;
9 (gdb)
10 14      auto lambda_function = [a = 0, &b](int c) mutable {a =
      c; b = a + c;};
11 (gdb) s
12 16      lambda_function(5);
13 (gdb) s
14 operator() (__closure=0x7fffffffd3c0, c=5) at ../lambdaas.cpp:14
15 14      auto lambda_function = [a = 0, &b](int c) mutable {a =
      c; b = a + c;};
16 (gdb) ptype /r lambda_function
17 type = struct {
18     int __a;
19     int &__b;
20 }
```

Лямбда с захватом - анонимная структура-функтор

```
1 (gdb) p &lambda_function
2 $1 = (struct {...} *) 0x7fffffffed3c0
3 (gdb) disassemble
4 Dump of assembler code for function operator()(int):
5     0x00005555555584f4 <+0>:      push    %rbp
6     0x00005555555584f5 <+1>:      mov     %rsp,%rbp
7     0x00005555555584f8 <+4>:      mov     %rdi,-0x8(%rbp)
8     0x00005555555584fc <+8>:      mov     %esi,-0xc(%rbp)
9 => 0x00005555555584ff <+11>:     mov     -0x8(%rbp),%rax
10    0x0000555555558503 <+15>:     mov     -0xc(%rbp),%edx
11    0x0000555555558506 <+18>:     mov     %edx,(%rax)
12    0x0000555555558508 <+20>:     mov     -0x8(%rbp),%rax
13    0x000055555555850c <+24>:     mov     (%rax),%ecx
14    0x000055555555850e <+26>:     mov     -0x8(%rbp),%rax
15    0x0000555555558512 <+30>:     mov     0x8(%rax),%rax
16    0x0000555555558516 <+34>:     mov     -0xc(%rbp),%edx
17    0x0000555555558519 <+37>:     add     %ecx,%edx
18    0x000055555555851b <+39>:     mov     %edx,(%rax)
19    0x000055555555851d <+41>:     nop
20    0x000055555555851e <+42>:     pop     %rbp
21    0x000055555555851f <+43>:     ret
22 End of assembler dump.
```

Generic lambdas (C++14 / C++20)

```
1 // C++14: auto параметр- → шаблонный operator()
2 auto adder = [](auto x, auto y) { return x + y; };
3 adder(1, 2);           // int → 3
4 adder(1.5, 2.5);      // double → 4.0
5
6 // C++20: явные шаблонные параметры + concepts
7 template <typename T, typename U>
8 concept Addable = requires(T t, U u) { t + u; };
9
10 auto typed = []<typename T, typename U>(T x, U y)
11     requires Addable<T, U> { return x + y; };
12
13 typed(1, 2);           // ok
14 // typed("a", 1);     // ошибка: нет Addable<const char*,int>
15
16 // Доступ к типу T по имени внутри тела:
17 auto front = []<typename T>(std::vector<T>& v) -> T& {
18     return v.front();
19 };
```

“Перегрузка” лямбд (overload trick)

```
1 // Наследование от нескольких лямбд (C++17)
2 template <typename... F>
3 struct overload : F... { using F::operator()...; };
4 template<class... Ts>
5 overload(Ts...) -> overload<Ts...>; // deduction guide
6
7 auto f = overload {
8     [](int i)      { std::cout << "int_" << i; },
9     [](double d) { std::cout << "double_" << d; },
10 };
11 f(42); f(3.14); // int 42 / double 3.14
12
13 // std::variant + overload ⚡ паттернматчинг-
14 std::variant<int, double, std::string> v = 3.14;
15 std::visit(overload {
16     [](int i)      { std::cout << i; },
17     [](double d)    { std::cout << d; },
18     [](std::string& s){ std::cout << s; }
19 }, v); // выведет: 3.14
```

std::function: стирание типов и рекурсия

```
1 // auto: нулевой overhead, конкретный тип лямбды
2 auto fast = [x = 5](int a) { return x + a; };
3
4 // std::function: type erasure, heap indirection
5 std::function<int(int)> flex = [x = 5](int a) {
6     return x + a;
7 }; // внутри - void* + виртуальный вызов
8
9 // Рекурсивная лямбда требует стирания типа:
10 // auto fact = [&fact](int n){...}; // ошибка: fact ещё неизвестен
11 std::function<int(int)> fact = [&fact](int n) {
12     return n <= 1 ? 1 : n * fact(n - 1);
13 };
14 fact(5); // 120
15 // Внимание: fact захвачена по ссылке - не перемещать!
16
17 // Deducing this
18 auto fact = [](this auto&& self, int n) -> int {
19     return n <= 1 ? 1 : n * self(n - 1);
20 };
21 fact(5); // 120
```